# APPLICATION OF DL AND ML TECHNIQUES TOWARDS A PREDICTION FRAMEWORK OF ACTIVE NETWORK TELESCOPE DATA

**Sivani U**

**Roll No. 20PD29**

DISSERTATION SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF
**FIVE YEAR INTEGRATED**
**M.Sc. DATA SCIENCE**
OF ANNA UNIVERSITY




**NOVEMBER 2023**

DEPARTMENT OF APPLIED MATHEMATICS AND COMPUTATIONAL SCIENCES

**PSG COLLEGE OF TECHNOLOGY**

(Autonomous Institution)

**COIMBATORE – 641 004**

**PSG COLLEGE OF TECHNOLOGY**

(Autonomous Institution)

**COIMBATORE – 641 004.**

**Seventh Semester**

**Project work**

# APPLICATION OF DL AND ML TECHNIQUES TOWARDS A PREDICTION FRAMEWORK OF ACTIVE NETWORK TELESCOPE DATA

Bona fide record of work done by

**Sivani U**

**Roll No. 20PD29**

Submitted in partial fulfillment of the requirements for the degree of

**FIVE YEAR INTEGRATED**

**M.Sc. DATA SCIENCE**

of Anna University

**NOVEMBER 2023**

**Academic Guide** **Head of the Department**

Submitted for the Viva-Voce Examination held on _____

**Internal Examiner** **External Examiner**

# CONTENTS

**ACKNOWLEDGEMENT** i

**SYNOPSIS** ii

**CHAPTER 1** 1

**INTRODUCTION** 1

**1.1 INSTITUTION PROFILE**....3

**1.2 SYSTEM ENVIRONMENT**....4

**1.3 TOOLS AND TECHNOLOGIES**....5

**1.4 GLOSSARY**....8

**1.5 LITERATURE SURVEY**....9

**CHAPTER 2** 14

**PREPROCESSING** 14

**2.1 DATA COLLECTION**....14

**2.2 MISSING VALUES**....17

**2.3 OUTLIER REMOVAL**....18

**2.4 SMOOTHING**....19

**2.5 NORMALIZATION**....20

**CHAPTER 3** 22

**MODEL SELECTION** 22

**3.1 ARIMA**....22

**3.2 RANDOM FOREST**....25

**3.3 RNN**....26

**3.4 LSTM**....30

**CHAPTER 4** 36

**RESULTS AND DISCUSSION** 36

**4.1 ANALYSIS**..... 36

**4.2 INFERENCE**.....47

**CONCLUSION** 48

**BIBLIOGRAPHY** 49

# ACKNOWLEDGEMENT

I am extremely grateful to **Dr. K. Prakasan,** Principal, PSG College of Technology, for giving me this opportunity to do my project at the Council of Scientific and Industrial Research Fourth Paradigm Institution, Bangalore.

I am deeply indebted to my internal guide **Dr. Shina Sheen,** Professor and Head**,** Department of Applied Mathematics and Computational Sciences, PSG College of Technology, Coimbatore for her continual support and ardent motivation.

I extend my profound gratitude to **Dr. R. Nadarajan**, Director, PSG Institute of Applied Mathematics and Computational Sciences for his able guidance throughout my project tenure.

I further extend my gratitude to my mentor **Dr. V. Anil Kumar**, Professor and Chief Scientist, Academy of Scientific and Innovative Research (AcSIR), Council of Scientific and Industrial Research Fourth Paradigm Institution (CSIR 4PI), at various phases of this project work.

I wish to acknowledge **Dr. HNV Dutt**, former Head of C-CADD, and **Dr. Ashapurna Marndi**, Principal Scientist, for their valuable contributions in making my work challenging and invaluable support in facilitating my learning curve.

Finally, I express my gratitude to all the faculty and staff of the Department of Applied Mathematics and Computational Sciences, PSG College of Technology, Coimbatore and my family and friends for their encouragement and support.

# SYNOPSIS

The autoregressive integrated moving average (ARIMA) models and their variants are in wide use for decades in the scientific field of time series forecasting, due to their mathematical simplicity and flexibility in application. There is a paradigm shift towards machine and deep learning approaches in most branches of knowledge requiring a time series prediction with the rapid advances in the development and efficient deployment of artificial intelligence models and techniques. It is a challenging task to assess the superiority of the new approach over the classic statistical algorithms.

One of the fertile topic in the time series analysis is the predictive behavior of IBR TCP traffic at a small aperture active network telescope, especially to examine the possibility of threat intelligence generation using this non-productive and seemingly harmless data.

Our work constitutes an extensive study of and the comparison of multiple models, including ARIMA, Random Forest, RNN, and LSTM, to gain valuable insights into the strengths and limitations of each approach in predicting packet counts within the network telescope domain.

The results emphasize the critical role of refined model architectures, optimizer selection, and hyper parameter tuning in achieving accurate and reliable predictions, underscoring the importance of leveraging sophisticated deep learning models for robust analysis within the framework of network telescope IBR data.

# CHAPTER 1

# INTRODUCTION

In the current digital landscape, cyber security has rapidly emerged as a preeminent global concern, garnering increasing attention from individuals, businesses, and governments alike. With the proliferation of interconnected devices, the expansion of digital infrastructures, and the escalating reliance on data-driven technologies, the vulnerabilities within cyberspace have become more pronounced and pervasive. The exponential growth of sophisticated cyber threats, ranging from data breaches and ransomware attacks to network intrusions and identity theft, has underscored the critical importance of robust cyber security measures on a global scale. As the digital realm continues to evolve and expand, the need for comprehensive and proactive strategies to safeguard sensitive information, protect privacy, and ensure the integrity of digital systems has become paramount. Addressing the multifaceted challenges posed by cyber security requires collaborative efforts, innovative solutions, and continuous advancements in technology and policy frameworks to effectively mitigate risks and fortify digital resilience across diverse sectors and communities worldwide.

In the dynamic landscape of cybersecurity, the intricate interplay of emerging threats and sophisticated attack vectors generates an unprecedented influx of data, posing a formidable challenge for security analysts and organizations. Amid this complex environment, network telescopes serve as a pivotal source of comprehensive data, offering valuable insights into the ever-evolving cybersecurity landscape. Operating as specialized monitoring systems, network telescopes capture and analyze a vast array of internet traffic, including anomalous and potentially malicious activities, providing crucial intelligence for threat detection and mitigation. By sifting through this massive influx of data, network telescopes facilitate the identification of patterns, trends, and emerging cyber threats, enabling proactive and effective cybersecurity measures. As an indispensable component of the cybersecurity ecosystem, network telescopes play a central role in fortifying digital infrastructures, enhancing resilience, and enabling preemptive strategies to safeguard against an array of cyber threats.

Network telescopes play a critical role in modern cybersecurity, serving as specialized monitoring systems designed to detect and analyze potentially malicious internet traffic. Figure 1.1 provides a general overview of a typical Network Telescope system, highlighting its key components and operational setup. The system includes allocated and assigned IPs shielded by a firewall, ensuring their isolation from the scope of Internet Background Radiation (IBR). The Network Telescope monitor, a robust server, is strategically configured to intercept and actively respond to darknet traffic, effectively exposing some of the potential security threats. Notably, this centralized server effectively manages all allocated but unassigned IPs, employing sophisticated mechanisms at the kernel, coding, or router level as shown in Figure 1.1.

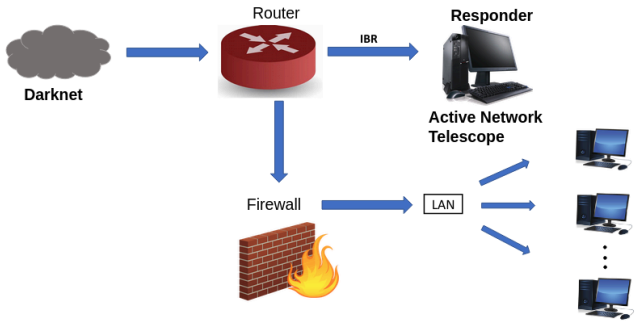



*Figure 1.1 General Network telescope setup*

To ensure comprehensive security, the database server responsible for storing raw TCP/IBR packets operates behind a firewall, safeguarding its data from unauthorized access. This strategic placement of the database server shields sensitive information from external vulnerabilities, thereby bolstering the overall security infrastructure. The forthcoming sections provide an in-depth exploration of the specific POC Network Telescope utilized in the current thesis, emphasizing its operational intricacies and the novel insights it offers for effective threat detection and network monitoring.

With the exponential growth of available data stemming from network responders, such as the network telescope, the application of data-driven models has gained critical significance. These models leverage the vast and diverse datasets generated by network telescopes, enabling sophisticated analysis and detection of potential security threats and anomalous network activities. By harnessing the wealth of information provided by these network responders, data-driven models offer unprecedented insights into complex network behaviors, facilitating

proactive threat detection, precise anomaly identification, and effective network monitoring. The utilization of data-driven models in conjunction with the wealth of data provided by network telescopes not only enhances the overall cybersecurity posture but also strengthens the capacity to anticipate and mitigate potential security breaches, ensuring robust protection for modern digital infrastructures. The current research work is to systematically build and finetune a model based prediction framework for the Internet Background Radiations and thereby forecast the arrival rate and pattern a priori.

## 1.1 INSTITUTION PROFILE

The Council of Scientific and Industrial Research Fourth Paradigm Institute (CSIR-4PI) in India is a distinguished research organization that embodies a commitment to advancing scientific knowledge and fostering innovation. Established as an autonomous body under the Council of Scientific and Industrial Research (CSIR), CSIR-4PI occupies a pivotal role in the realms of mathematical modeling, computer-based simulations, and data-driven research. The institute, situated within the sprawling campus of the CSIR National Aerospace Laboratories (CSIR-NAL) in Bengaluru, is at the forefront of pioneering research spanning diverse scientific disciplines.

CSIR-4PI operates with a clear mission to cultivate a culture of mathematical modeling and data-driven research across the CSIR network, aimed at accelerating the pace of discovery in fields of significant interest. The institute pursues this mission through a comprehensive two-fold approach, emphasizing the critical domains of data science and supercomputing, and the expansion of data science research to address strategic and societal challenges. These endeavors involve forging industry collaborations and deploying advanced computational tools across various domains, with a recent focus on deep learning-based time series analysis in the realm of earth sciences.

CSIR-4PI collaborates closely with the Academy of Scientific & Innovative Research (AcSIR), playing a pivotal role in nurturing the next generation of researchers and professionals in the fields of science and engineering. Leveraging CSIR's extensive infrastructure and scientific resources, AcSIR offers a comprehensive educational framework, fostering innovation and interdisciplinary integration. The academy's commitment to providing degrees and conducting

research in cutting-edge areas of science and technology serves as a testament to its dedication to excellence in scientific education and research.

The Student Programme for Advancement in Research Knowledge (SPARK), an integral component of CSIR-4PI, offers a unique platform for exceptional and motivated students from renowned universities. Through SPARK, students actively participate in significant research projects and thesis work, gaining valuable experience in mathematical modeling and the simulation of complex systems. The program promotes a collaborative environment, fostering partnerships between students, CSIR-4PI scientists, and academic faculty members, thus laying the foundation for enduring research collaborations.

CSIR-4PI stands as a vanguard of scientific research and innovation, dedicated to resolving intricate challenges and advancing knowledge in the digital era. With its unwavering focus on data science, supercomputing, cyber security and interdisciplinary research, the institute is poised to make substantial contributions to India's scientific landscape and global scientific advancements.

The active network telescope available at CSIR 4PI is a sophisticated and specialized system designed to actively monitor and analyze network traffic. Unlike passive network telescopes that only observe and record incoming data, active network telescopes engage, in a light-weight manner, the incoming TCP SYN packets, and respond with appropriate SYN/ACK packet to successfully establish a three-way handshake. This active engagement enables the network telescope to verify the source IP address of the IBR originator and make source that the packet is free from source IP address spoofing.

## 1.2 SYSTEM ENVIRONMENT

This section provides an overview of the system environment employed during the development of the project. It outlines the technological infrastructure, software frameworks, and hardware configurations utilized to facilitate the design, implementation, and testing processes.

### 1.2.1 Hardware Configuration

**Processor** : Intel® Xeon® Silver 4314, 32 cores

**Hard Disk** : 48.0 TB in a RAID configuration

**RAM** : 64.0 GB

### 1.2.2 Software Configuration

**Operating System** : Fedora Linux 36

**Languages** : Python

**Tools** : Google Colab

## 1.3 TOOLS AND TECHNOLOGIES

The tools and technologies utilized in the development of the prediction model for the network telescope constitute a powerful arsenal of resources. Leveraging the versatility of Python, alongside libraries like NumPy and Pandas, facilitated seamless data handling and analysis. Plotly's interactive visualization capabilities enhanced data comprehension, while Scikit-learn and Keras enabled the integration of robust machine learning and deep learning techniques. Additionally, SciPy's numerical algorithms and TensorFlow's efficient computation framework bolstered the model's scientific computing and predictive capabilities. Collectively, these tools and technologies synergized to create a sophisticated and efficient network telescope prediction model, poised to address complex challenges in network security and analysis.

### 1.3.1 Python

Python, introduced by Guido van Rossum in the early 1990s, has widespread recognition for its elegance, simplicity, and adaptability. Python offers clean syntax and readability, enabling programmers to express concepts with clarity and conciseness. Notably, Python accommodates multiple programming paradigms, including object-oriented, imperative, and functional programming, allowing developers the freedom to address diverse project requirements. Its comprehensive standard library empowers users to handle an extensive spectrum of tasks, spanning from web development and data analysis to scientific computing and artificial intelligence. Python's cross-platform compatibility facilitates seamless code deployment across various environments, enhancing its suitability for a myriad of applications. Bolstered by an

active and collaborative community, Python has solidified its status as a premier programming language, finding extensive application across diverse industries and domains.

### 1.3.2 Numpy

Powerful Python libraries like NumPy are the cornerstones of scientific computing and data handling. Large, multidimensional arrays and matrices can be supported, and a wide range of sophisticated mathematical operations can be performed effectively on these arrays. NumPy has developed into a crucial tool for numerical computing in Python because of to its reliable data structures and a wealth of routines for quick array computations, such as mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, and statistical operations. It is the proposed option for activities requiring numerical computations, data analysis, and machine learning. Because of its ease of use, effectiveness, and adaptability, it has become a pillar of the Python ecosystem for scientific computing and data analysis.

### 1.3.3 Pandas

Python has a robust open-source data analysis and manipulation toolkit called Pandas. It is renowned for its effectiveness and simplicity and offers simple data structures and data analysis tools, making it a crucial tool for tasks involving data wrangling. Users may simply do activities like data indexing, selection, filtering, grouping, merging, reshaping, and more with the help of Pandas, which offers smooth management of labeled and relational data. Additionally, Pandas interfaces easily with other Python packages, facilitating thorough processes for data processing and analysis. Data scientists, analysts, and researchers who work with structured data favor it because of its flexibility, speed, and wide features.

### 1.3.4 Plotly

Plotly is a strong and flexible Python module that makes it easier to create interactive and publishable data visualizations. Scattered plots, line plots, bar charts, heatmaps, and other graphing features are available. Plotly gives customers the ability to create highly configurable

plots with features like zooming, panning, and hovering for in-depth data study thanks to its user-friendly interface and extensive customization choices. Additionally, Plotly can be smoothly connected with a number of Python frameworks, including Jupyter Notebooks, Flask, and Dash, making it easy to build dynamic and interactive web apps.

### 1.3.5 Sklearn

A notable open-source machine learning package for the Python programming language is scikit-learn. This potent toolkit, which is based on the Python libraries NumPy, SciPy, and Matplotlib, provides a variety of supervised and unsupervised learning techniques, making it an invaluable resource for tasks involving data analysis and machine learning. Sklearn makes it easier to employ a variety of machine learning techniques, such as classification, regression, dimensionality reduction, model selection, and preprocessing, owing to its user-friendly interface and thorough documentation. Its versatility and interoperability are increased by the seamless integration that its modular design enables with other Python libraries and frameworks.

### 1.3.6 Keras

Deep learning library created in Python, called Keras, is open-source. It is well known for its user-friendly, modular, and simplistic interface, making it one of the most well-liked options for neural network rapid prototyping. A high-level API offered by Keras makes it much easier to create and experiment with deep learning models. Because of its simple and adaptable design, it makes building complex neural networks simple, making it a useful tool for both new and seasoned researchers. With an emphasis on simplicity and extensibility, Keras makes it simple to build convolutional neural networks (CNNs), recurrent neural networks (RNNs), and hybrids of the two, enabling users to create and implement cutting-edge deep learning solutions with relative ease.

### 1.3.7 SciPy

SciPy is an open-source Python library that builds upon the NumPy package, providing a wide array of numerical algorithms and tools for scientific and engineering applications. It serves as an essential resource for scientific computing, enabling efficient and convenient solutions for

tasks such as optimization, integration, interpolation, signal processing, linear algebra, and more. With its various submodules, including scipy.optimize for optimization tasks, scipy.integrate for numerical integration, scipy.signal for signal processing, and scipy.linalg for linear algebra operations, SciPy offers a comprehensive and powerful suite of tools for data analysis and manipulation. Moreover, its integration with other Python libraries, such as Matplotlib for visualization and Pandas for data manipulation, facilitates seamless data analysis workflows. The library's extensive documentation and active community support make it an invaluable asset for researchers, scientists, and engineers seeking to leverage Python for complex scientific computations and data analysis tasks.

### 1.3.8 Tensorflow

TensorFlow is an open-source machine learning framework renowned for its efficiency and versatility in building and deploying machine learning models. Developed by the Google Brain team, it has become one of the most popular libraries for deep learning tasks. At its core, TensorFlow operates through a data flow graph, where nodes represent mathematical operations, and edges represent the multidimensional data arrays, known as tensors, that flow between them. This structure allows for the seamless parallel execution of operations across multiple devices, including CPUs and GPUs, enhancing its capacity for large-scale computations. TensorFlow's extensive collection of high-level APIs simplifies the process of constructing neural networks. With its ability to handle complex computations and its compatibility with various platforms, including mobile devices, TensorFlow remains a crucial tool for research, development, and production deployment of machine learning and deep learning models.

## 1.4 GLOSSARY

The glossary section serves as a key reference point within this body of work, providing concise definitions of specific terms relevant to the domain at hand. These terms have been carefully curated to elucidate critical concepts integral to the understanding and analysis of various aspects within the realm of cybersecurity and data analysis. Through the defined terms, a comprehensive grasp of fundamental concepts such as Internet Background Radiation (IBR), Non-Stationary Data, and Active Network Telescopes, enable to navigate complex discussions and delve deeper into the intricate workings of digital networks and security protocols.

### 1.4.1 IBR

Internet Background Radiation (IBR) refers to a specific category of internet traffic that differs from regular user interactions. It encompasses typical packets that traverse the entire cyberspace. These packets do not originate from conventional user behaviors but instead constitute a diverse range of digital "background noise." Although they may seem harmless, they often contain valuable information about potential cyber threats and undesired activities occurring on the internet. Analyzing IBR can provide crucial insights for identifying and mitigating potential security risks and maintaining the integrity of digital networks.

### 1.4.2 Non-Stationary data

Non-stationary data refers to a time series where the mean and variance exhibit fluctuations over time, signifying the presence of a discernible trend. Such data may also demonstrate varying seasonal patterns, leading to fluctuations at specific intervals. Moreover, the absence of a clear pattern characterizes the data's seemingly random behavior, making it challenging to predict future trends accurately.

### 1.4.3 Active network telescope

An active network telescope serves as a specialized network monitoring system that captures and analyzes unsolicited or suspicious network traffic. By utilizing inactive or unused IP addresses, it detects and examines potentially malicious activities such as scanning, probing, and malware dissemination attempts. This technology plays a crucial role in identifying and understanding cybersecurity threats, including distributed denial-of-service (DDoS) attacks and other forms of network-based malicious behavior, thus aiding in the development of effective security measures and countermeasures.

## 1.5 LITERATURE SURVEY.

The exploration of Machine Learning and Deep Learning methods within the domain of network telescope data has witnessed significant advancements, particularly in understanding and mitigating cyber threats, with a specific focus on TCP port scanning. Notably, prior research efforts have primarily utilized passive network telescopes to study Internet Background

Radiation (IBR), shedding light on potential vulnerabilities and malicious activities targeting both active and dormant IP addresses. Researchers have leveraged various data analysis and machine learning techniques to discern distinct patterns of scanning behaviors, classify attackers strategies, and develop robust threat detection systems.

Several seminal studies have delved into the realm of network telescope data analysis, addressing a diverse range of challenges and security concerns. Key contributions include the early groundwork by Moore et al. (2004), emphasizing "backscatter analysis" to comprehend network security events, and the subsequent investigations by Yu Jin et al. (2007) that centered on identifying outside scanners and their scanning patterns in the IP gray space. Additionally, Félix Iglesias and Tanja Zseby proposed a lightweight method to distinguish different attack types in an IP darkspace monitor, while Farooq Shaikh et al. focused on classifying unsolicited Internet of Things (IoT) devices using supervised machine learning algorithms.

Furthermore, Nipun Ramakrishnan and Tarun Soni explored the application of Recurrent Neural Network (RNN) architectures for network traffic prediction, while Sanjay Kumar et al. introduced a threat detection system based on Machine Learning classifiers trained using darknet traffic. Olivier Cabana et al. delved into threat intelligence generation using network telescope data for Industrial Control Systems (ICSs), while Michalis Kallitsis et al. developed a framework for analyzing darknet data to provide actionable insights for security analysts. Lastly, Almomani. A. introduced a darknet traffic analysis based on modified stacking ensemble learning algorithms, emphasizing the system's efficacy in distinguishing between malignant and benign traffic. Collectively, these studies represent a comprehensive exploration of network telescope data analysis, emphasizing the critical role of Machine Learning and Deep Learning methodologies in fortifying cybersecurity frameworks.

**Moore et al. (2004)** [**1]** were some of the first researchers to actually study IBR, referring to it as “backscatter analysis”, to view remote network security events such as various forms of flooding denial-of-service attacks, infection of hosts by Internet worms, and network scanning.

**Yu Jin, et al. (2007)** [**2**] have used a month-long netflow traffic trace, based on collection of IP addresses within their campus network which are not assigned to any "active" host, calling it IP gray space, to identify and extract potential outside scanners and their associated activities. Further they have applied data mining and machine learning techniques to analyse the scanning patterns of these scanners and classify them as focused hitters, random address scanners, and block wise scanners.

**Félix Iglesias and Tanja Zseby [3]** have proposed a lightweight method to distinguish different attack types observed in an IP darkspace monitor and is based on entropy measures of traffic-flow features and machine learning techniques. They conclude that sudden changes in traffic types caused by new attacks or attack preparation activities can be identified based on entropy variations.

**Farooq Shaikh et. al. [4]** address the problem of classification of unsolicited Internet of Things [IoT] devices in enterprises using machine learning (ML), using IP header information from darknet data collected. They consider multiple supervised ML algorithms to classify these Layer 3 headers and conclude that their model can be used by enterprises as a part of their intrusion detection system to quickly identify infected IoT devices within their own environment as well as identify scanning activities directed towards them.

**Nipun Ramakrishnan and Tarun Soni [5]** propose different Recurrent Neural Network (RNN) architectures, viz., the standard RNN, Long Short Term Memory (LSTM) networks, and Gated Recurrent Units (GRU) to solve the network traffic prediction problem. They use the detailed real-life traffic measurement data of the GEANT/Abilene backbone networks. They analyze the performance of these models on the following problems in network traffic prediction: volume prediction, packet protocol prediction, and packet distribution prediction. They conclude that RNN architectures demonstrate promising results in all three of these domains in network traffic prediction, outperforming standard statistical forecasting models significantly.

**Sanjay Kumar et. al. [6]** have proposed a threat detection system based on Machine Learning classifiers that are trained using darknet traffic. The data used by them is the darknet traffic data collected from the network sensors deployed at SURFnet, for extracting network traffic features and finding threat patterns. They present a framework that uses supervised machine learning and

a concept drift detector and demonstrate that their classifiers can distinguish between benign and malign traffic.

**Olivier Cabana et. al. [7]** deal with threat intelligence generation using network telescope data for Industrial Control Systems (ICSs), which are fertile ground for the threat actors due to the scale of damages it can create. Often the threat actor's carryout reconnaissance campaigns, which aim at vulnerable ICSs, some portion of which is directed to network telescopes.

They use machine learning to classify the sources behind the campaigns and identify threat actors such as botnets, malicious attackers, or researchers. In this regard, they have used 12.85 TB of network traffic received by our observed darknet IP space over a span of 330 days.

**Michalis Kallitsis et. al. [8]** considers the aspect of analyzing the darknet data which can provide actionable insights to security analysts that can be used to prevent or mitigate cyber-threats. Large Darknets, however, observe millions of nefarious events on a daily basis which makes the transformation of the captured information into meaningful insights challenging. Their proposed framework: (i) Extracts a high dimensional representation of Darknet events composed of features distilled from Darknet data and other external sources; (ii) Learns, in an unsupervised fashion, an information-preserving low-dimensional representation of these events (using deep representation learning) that is amenable to clustering; (iv) Performs clustering of the scanner data in the resulting representation space and provides interpretable insights using optimal decision trees; and (v) Utilizes the clustering outcomes as "signatures" that can be used to detect structural changes in the Darknet activities. They evaluate the proposed system on a large operational Network Telescope and demonstrate its ability to detect real-world, high-impact cybersecurity incidents.

**Almomani. A. [9]** presents a darknet traffic analysis based on new machine learning classifiers known as modified stacking ensemble learning algorithms**.** In dealing with darknet attack problems, this new system uses predictions formed by 3 base learning techniques. They have tested their framework on a dataset with more than 141,000 records analyzed from CIC-Darknet 2020. The experimental results have demonstrated the study's classifiers' ability to distinguish

between the malignant traffic and benign traffic easily. They have also noted that their proposed system has the best standard deviation compared with current AI algorithms.

The literature survey reveals significant strides in leveraging machine learning and deep learning techniques for network telescope data analysis, with a primary focus on mitigating cyber threats, particularly TCP port scanning. While existing studies have made valuable contributions in understanding scanning behaviors, classifying attacker strategies, and enhancing threat detection, there are notable limitations. Some works address specific aspects like IoT device threat detection without offering a comprehensive view of packet count prediction. Moreover, challenges persist in adapting models to evolving patterns and dynamic network telescope environments. This research aims to bridge these gaps by conducting a thorough evaluation of diverse models, including ARIMA, Random Forest, RNN, and LSTM, for predicting packet count in network telescope data. Through a holistic approach encompassing residual analysis, optimization algorithms, and model configurations, this study intends to provide nuanced insights and propose refinements to enhance predictive accuracy in the ever-changing landscape of network telescope data.

# CHAPTER 2

# PREPROCESSING

This section navigates through critical steps in data collection and preprocessing for network telescope data analysis in the cybersecurity context. The process involves capturing TCP packets through a three-way handshake protocol, utilizing Wireshark for detailed analysis. Handling missing values employs mean filling, and outlier management recognizes the need for careful consideration of extreme values. Smoothing techniques, particularly moving average smoothing, aim to refine packet count data for improved accuracy. The crucial step of normalization, focusing on Min-Max normalization, ensures standardized data for efficient model training, mitigating outliers impact. Together, these methods form a robust foundation for accurate insights into network activity and security threats.

## 2.1 DATA COLLECTION

In CSIR 4PI, the approach to store data of a network telescope is by single logical volume of large storage capacity. Here the data which are TCP packets is collected after completion of three-way handshake process where the client represents a darknet causing Internet Background Radiation (IBR) and the server functions as an active responder known as a network telescope. The three-way handshake protocol assumes a pivotal role in mitigating spoofing and DDoS backscatters traffic entering into the data pool. In this configuration, the darknet operates as a covert network observing unauthorized traffic, while the network telescope acts as a responsive entity, intercepting and analyzing incoming connections. The three-way handshake initiates with the darknet sending a SYN packet, followed by the network telescope responding with a SYN-ACK packet, and the darknet ultimately acknowledged with an ACK

packet as shown in Figure 2.1.

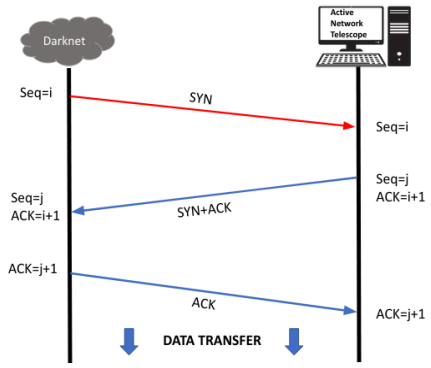



*Figure 2.1 3-Way handshake protocol*

The `.pcap` file format serves as a repository for network packet data, encompassing diverse traffic types, including those based on TCP (Transmission Control Protocol), UDP (User Datagram Protocol), and an array of other network protocols. When examining these files, Wireshark emerges as a prominent and robust network protocol analyzer, proficient in decoding and presenting the contents of `.pcap` files. Through Wireshark, users can thoroughly scrutinize the intricate details of network traffic and conduct granular analyses at the level of individual data packets.

A typical dataset within a '.pcap' file encompasses a range of crucial information, including source and destination IP addresses, source and destination port addresses, and various flags such as SYN, ACK, PSH, FIN, and more, which carry significant implications for network communication. Additionally, the payload section captures a plethora of data forms, spanning audio, video, text, hyperlinks, potentially malicious URLs, links to download malicious files, as well as specific patterns that might correspond to Mirai botnet packets or other suspicious activities. To ensure efficient data management and accessibility, these diverse packet details are systematically stored within a MySQL database, enabling robust querying capabilities.

In predicting potential anomalous activities worldwide, an SQL query is utilized to extract relevant data from the database. The data encompasses critical features, including a 'timestamp' denoting the occurrence date of daily malicious activities in the format of day, month, and year. The 'country count' denotes the collective tally of countries implicated in these malicious activities, facilitated by leveraging the geolocation information of IP addresses through

the use of the comprehensive database provided by 'IP2Location.' This database furnishes essential geographic details such as country, region or state, city, latitude, longitude, ZIP code, and time zone.

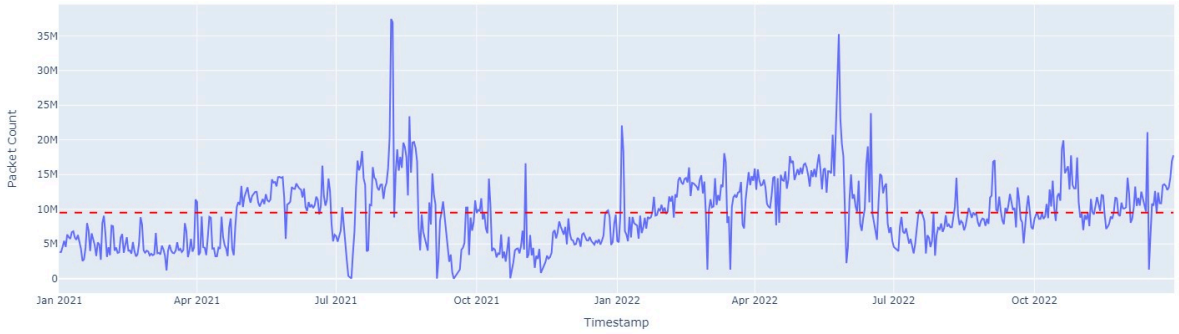



*Figure 2.2 illustrates the relationship between time and packet count for all ports during the years 2021 and 2022.*

Furthermore, the 'IP count' indicates the quantity of IP addresses generating Internet Background Radiation (IBR) targeting the designated pool of IPs. The 'ASN count' signifies the aggregate count of Autonomous System Numbers (ASNs), which serve as unique identifiers assigned by regional Internet registries to facilitate the exchange of routing information between distinct Autonomous Systems on the internet.

Similarly, the 'ASO count' encompasses the collective count of Autonomous System Organizations (ASOs) responsible for managing and maintaining routing policies and protocols within their respective Autonomous Systems. Additionally, the 'proxy count' aggregates the incoming proxy instances, i.e, the number of connections that arrived through a in between proxy server, rather than direct transmission. Further number of total packets received in a day is enclosed in the packet count feature.

In the initial phase of our research, we commenced data collection by capturing one month of network traffic data from specific high-traffic ports, including port 22 (SSH), port 23 (Telnet), port 80 (HTTP), and port 443 (HTTPS). Subsequently, we expanded our investigation to encompass a comprehensive one-year time frame, spanning from the previous year to the current date, incorporating the years 2021 and 2022 as shown in Figure 2.2. Building upon the insights gleaned from this expanded dataset, we further extended our analysis to encompass all ports, facilitating a more holistic understanding of the network activity across diverse communication

channels.

In pursuit of a comprehensive and up-to-date analysis, we continued to accumulate data beyond 2022, capturing network traffic from the intervening period up to the present moment. By progressively broadening the scope of our data collection efforts, we have endeavored to provide a more nuanced and detailed perspective on the evolving patterns and trends in cyberspace.

## 2.2 MISSING VALUES

Due to a variety of uncertain factors, the capture and storage of packets within the database have been impacted, resulting in the failure to retain records of potentially malicious instances of Internet Background Radiation (IBR). These uncertainties arise from a multitude of operational challenges, including intermittent network inconsistencies, hardware malfunctions, power supply disruptions, server downtime, filesystem full and insufficiencies within the database storage, necessitating vigilant monitoring.

The complexity of network infrastructure, coupled with potential hardware instabilities, has intermittently led to the unavailability of data storage resources, thereby obstructing the seamless recording and preservation of critical packet information. Moreover, periodic system interruptions and unforeseen process terminations have compounded the challenges, leading to the unintended cessation of data retention operations, impeding the ability to comprehensively monitor and analyze potential instances of malicious cyber activities characteristic of IBR.

The missing data over the time period from 2021-01-01 to 2022-12-31 is 2021-07-10, 2021-08-28, 2021-08-29, 2021-09-05, 2021-09-10, 2021-09-11, 2021-09-12, 2021-09-17, 2021-09-18, 2021-09-19, 2021-10-24, 2021-11-13, 2021-11-14, 2022-07-02, 2022-07-03, 2022-12-15

The missing data is replaced with the mean filling method, a technique used to impute missing values in a dataset by replacing them with the mean of the available data. Filling missing values with the mean helps maintain the overall distribution and central tendency of the data, preventing significant shifts in the dataset's characteristics. The mean filling method is straightforward and

easy to implement, making it a convenient choice for quick data preprocessing. Its simplicity allows for rapid application across large datasets, saving time and effort during the data cleaning process. By replacing missing values with the mean, the mean filling method helps minimize the bias that may arise from the presence of missing data. This can lead to more accurate and reliable statistical analyses and modeling, as the imputed values are based on the general trend of the available data. The mean filling method can be applied to both numerical and categorical data, providing a versatile solution for handling missing values in different types of datasets without the need for complex transformations or adjustments.

## 2.3 OUTLIER REMOVAL

An outlier denotes an observational data point that markedly deviates from the general pattern exhibited by other data within a dataset. This deviation is often represented by an exceptionally large or small value in relation to the overall data distribution. The emergence of outliers can be attributed to several factors, including errors during data entry, discrepancies in experimental procedures, or the occurrence of authentic yet infrequent events.

By eliminating outliers, statistical analyses and models remain robust and accurate, preventing distorted findings. This process sustains the consistency and validity of drawn conclusions, reducing the potential for misleading results from aberrant data points. Moreover, outlier identification and elimination enhance model effectiveness, enabling precise and reliable insights for informed decision-making.

In the process of data analysis, particularly in dealing with datasets that exhibit significant deviation and outliers, it is imperative to exercise caution. While these outliers may introduce considerable variability, it is essential to acknowledge their significance. These extreme spikes represent actual and valid values, signifying substantial packet receptions by the network telescope.

Adhering to domain expertise, it becomes apparent that completely disregarding these values would lead to an incomplete understanding of the underlying data dynamics. Hence, an approach that carefully integrates these outlier values within the analysis framework is crucial for ensuring a comprehensive and accurate interpretation of the dataset. This holistic perspective allows for a

more nuanced and insightful comprehension of the underlying patterns and behaviors within the dataset, leading to more informed decision-making and robust analytical outcomes.

## 2.4 SMOOTHING

Smoothing is a fundamental data preprocessing technique used to reduce noise and fluctuations within datasets, particularly those with continuous outputs. By applying smoothing techniques, we can extract underlying trends and patterns from noisy data, facilitating clearer insights and more accurate predictions.

In the context of continuous output prediction, the significance of smoothing arises from its ability to manage irregularities and fluctuations within datasets. These irregularities, often in the form of spikes or sudden fluctuations, can distort the interpretation of data and hinder the accuracy of predictive models. Smoothing helps mitigate these issues, enabling a more refined analysis and ensuring that the underlying trends are easily discernible.

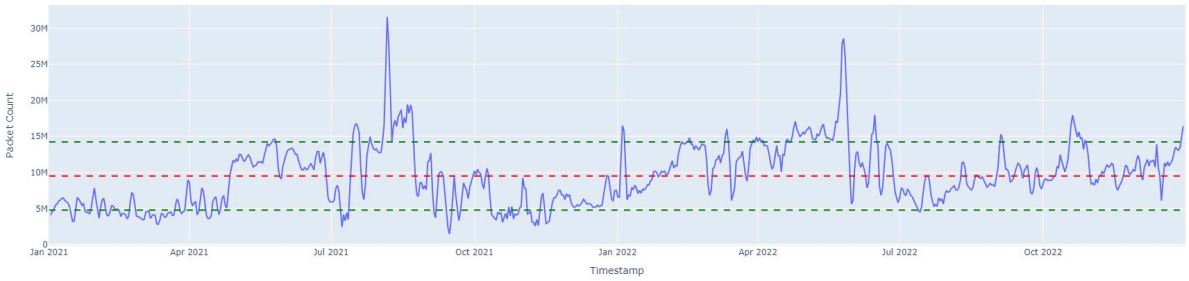



*Figure 2.3 illustrates smoothed representation between time and packet count for all ports of years 2021 and 2022.*

Smoothing offers numerous advantages, contributing to the reliability and accuracy of data analysis and prediction models. Some key advantages include noise reduction, enhanced predictive capabilities, and improved visualization of underlying trends, enabling better-informed decision-making and more accurate predictions.

The moving average smoothing technique is applied with a window size of 3 as shown in Figure 2.3. This technique involves replacing each data point with the average of its neighboring points within the specified window. By implementing this technique for 'packet_count' data, the code effectively smoothens the data, reducing the impact of spikes and high deviations. The chosen window size of 3 is optimal for balancing trend capture and over-smoothing, preserving the

original data characteristics while reducing noise.

For network telescopes, the application of smoothing techniques is critical. Given high deviation and spikes in packet count information, the accurate identification of genuine network activity patterns and potential security threats can be challenging. Smoothing aids in distinguishing anomalies from regular patterns, thereby enhancing the network telescope's ability to detect and mitigate potential security threats effectively.

Overall it plays a pivotal role in data analysis and prediction, enabling a clearer understanding of underlying trends and patterns. In the case of packet count prediction in a network telescope, smoothing is indispensable for reducing the impact of fluctuations and deviations, leading to more accurate and reliable insights into network activity and potential security threats.

## 2.5 NORMALIZATION

Normalization is a crucial data preprocessing technique that transforms data into a standardized format, ensuring that the data is within a specific range. It is commonly used in machine learning, data mining, and various statistical analyses. The primary goals of normalization are to scale the data to a standard range, prevent biases in the model, and improve the model's performance. Normalization also aids in speeding up the convergence of machine learning algorithms, enabling more efficient training and better performance.

In the context of model performance, normalization helps in improving the convergence rate and efficiency of various optimization algorithms, particularly in cases where features have different ranges. By bringing all features to a similar scale, normalization ensures that no particular feature dominates the learning process due to its larger magnitude. This balance in feature scales enables the model to give equal importance to all features during the training phase, leading to better model performance, improved accuracy, and faster convergence.

Min-Max normalization, a specific type of normalization, rescales the data to a fixed range, typically between 0 and 1. This method is beneficial when there are high deviations or spikes in the data, as it can effectively handle outliers and preserve the original distribution of the data. It

is particularly useful when the underlying data distribution is not Gaussian and the data range is known. Moreover, Min-Max normalization retains the relationships between different data points, making it a suitable choice when preserving the relationships among data values is essential for the analysis or modeling process. The formula is given as,

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

***(1)***

Here, $X_{norm}$ is the normalized value, X is the original value, $X_{min}$ is the minimum value in the feature, and $X_{max}$ is the maximum value in the feature.

In the case of our network responder dataset with high deviation and spikes in the packet count information, using Min-Max normalization can effectively scale the data to a specific range, thereby mitigating the impact of outliers and variations in the data. By ensuring that the packet count information is transformed to a standardized scale, the Min-Max normalization method can help in achieving a more consistent and reliable analysis, enabling better insights and more accurate modeling results.

# CHAPTER 3

# MODEL SELECTION

Model selection is a fundamental process in building reliable and accurate predictive models for network telescopes. It involves rigorously testing multiple models and selecting the one that demonstrates superior predictive performance based on specific criteria. By evaluating various models on a given dataset using appropriate metrics and validation techniques, we ensure that the chosen model can generalize effectively to new and unseen data. In this chapter, we have executed model selection across statistical, machine learning, and deep learning models, emphasizing the criticality of thorough evaluation and comparison to make informed decisions about the most suitable model for the task.

## 3.1 ARIMA

A statistical model is a formal representation of a set of assumptions concerning the generation of observed data. It is a mathematical framework that describes the relationships between different variables and enables us to make inferences or predictions about a population based on a sample of data. Statistical models are used to understand the underlying structure in data, identify patterns, and make predictions or estimations about future observations. Examples of statistical models include linear regression models, logistic regression models, time series models like ARIMA, and various types of Bayesian models.

ARIMA is a widely used statistical method for analyzing and forecasting time series data. It is a combination of autoregressive (AR) and moving average (MA) models along with differencing to handle non-stationary data. The acronym ARIMA stands for AutoRegressive (AR) which considers the relationship between the current observation and previous observations. It indicates that the current value of the time series depends on its own past values, Integrated (I) involves differencing the raw observations to make the time series stationary, which essentially means that

the mean, variance, and autocorrelation structure do not depend on time. A stationary time series is easier to model and forecast, Moving Average (MA) deals with the relationship between the current observation and a residual error from a moving average model applied to lagged observations.

ARIMA models are typically denoted as ARIMA (p, d, q), where:

1. p is the order of the autoregressive part.
2. d is the degree of differencing (the number of times the data have had past values subtracted).
3. q is the order of the moving average part.

Normalization is not typically required for time series data when using the ARIMA (AutoRegressive Integrated Moving Average) model. ARIMA focuses on understanding the temporal patterns and autocorrelations within the time series data.

The Evaluation of the ARIMA model and further analysis is performed with residual analysis plots. Residual analysis in ARIMA modeling is a fundamental process for evaluating the performance and validity of the model. Through various graphical tools like scatter plots, density plots, autocorrelation plots, and Q-Q (quantile-quantile) plots, residual analysis offers a comprehensive understanding of the discrepancies between the actual and predicted values. These plots provide insights into the presence of patterns, trends, outliers, and the normality assumption of the residuals, aiding in the identification of potential model deficiencies and the necessary adjustments for enhanced forecasting accuracy.

The Ljung-Box test is a statistical test used to check whether a set of autocorrelations of a time series are different from zero. In simpler terms, it helps to determine whether there is evidence of autocorrelation in a dataset, which is important in understanding the underlying patterns and dependencies in time series data. It is commonly used in time series analysis to assess the quality of a model by examining whether the residuals (the differences between observed and predicted values) exhibit significant autocorrelation. The Ljung-Box test statistic is calculated using the following formula:

$$Q = n(n + 2) \sum_{k=1}^{h} \frac{r_k^2}{n-k} \quad \textbf{(2)}$$

where:

- $Q$ is the test statistic.
- $n$ is the number of observations in the time series.
- $h$ is the number of lags being tested.
- $r_k$ is the sample autocorrelation at lag $k$.

The Durbin-Watson test is a statistical test used to check for the presence of autocorrelation in the residuals of a regression analysis. It provides a way to detect whether there is a correlation between consecutive residuals in a time series or panel data analysis. The test statistic ranges from 0 to 4, with values around 2 indicating no autocorrelation. A value significantly below 2 suggests positive autocorrelation, while a value significantly above 2 suggests negative autocorrelation. The formula for the Durbin-Watson test statistic is as follows:

$$DW = \frac{\sum_{t=2}^{T} (e_t - e_{t-1})}{\sum_{t=1}^{T} e_t^2} \quad \textbf{(3)}$$

where:

- $e_t$ represents the residuals or errors obtained from the regression analysis at time t,
- $T$ denotes the total number of observations in the dataset.

Linearity is an essential assumption in linear regression and time series analysis, including ARIMA modeling. When discussing linearity in the context of ARIMA models and residual analysis, it generally refers to the linear relationship between the independent and dependent

variables. If the relationship is not linear, it can affect the model's performance and the interpretation of the diagnostic tests, such as the Ljung-Box test and the Durbin-Watson test.

When considering linearity and its relation to the diagnostics obtained, it's important to assess how the residuals behave concerning the linear relationship. The Ljung-Box test and the Durbin-Watson test focus on checking the presence of autocorrelation, which is related to the linear relationship between the dependent variable and its lagged values. If the residuals exhibit signs of autocorrelation, it might indicate a violation of the linearity assumption.

It's crucial to inspect the scatter plots of the residuals against the predicted values or the independent variable to visually assess linearity. If the scatter plots show patterns, trends, or non-random distributions, it might suggest nonlinearity in the data, which can affect the model's performance.

## 3.2 RANDOM FOREST

The Random Forest Regressor is a type of ensemble learning method widely used in machine learning for regression tasks. Unlike individual decision trees, which are susceptible to overfitting and variance, the Random Forest Regressor leverages a collection of decision trees to enhance predictive accuracy and generalization. This ensemble learning technique works by training numerous decision trees on random subsets of the training data. Each tree is built independently, incorporating different subsets of features at each node, thus introducing a level of randomness that prevents dominant features from overshadowing others. The process of aggregating predictions from each tree helps minimize overfitting and promotes robust predictions. Furthermore, the Random Forest Regressor is known for its flexibility in handling large datasets and its resilience to noisy data

Random Forest Regressor and ARIMA (AutoRegressive Integrated Moving Average) are two distinct modeling approaches used for time series forecasting and regression tasks, each with its unique strengths and applications. While ARIMA is a statistical method that primarily focuses on capturing the linear relationships and autocorrelation within the data, the Random Forest Regressor is an ensemble learning technique that leverages the power of multiple decision trees to provide robust predictions. The Random Forest Regressor offers greater flexibility, robustness,

and adaptability to complex datasets compared to the more traditional statistical method ARIMA. Its ability to handle non-linear relationships, noise, and large feature spaces makes it an attractive choice for predictive modeling tasks that involve intricate and multidimensional datasets, where traditional statistical methods may fall short.

The Random Forest Regressor automates feature selection by assessing the importance of various features during the construction of decision trees within the ensemble. By prioritizing influential features based on their contribution to predictive power, the algorithm effectively filters out noise and irrelevant attributes. This automated mechanism enhances predictive accuracy and facilitates a deeper understanding of complex relationships within the network telescope data.

The decision to select the Random Forest Regressor was based on its ability to handle high-dimensional datasets with diverse features, robustness to noise and outliers, automated feature selection, ensemble learning approach, and computational efficiency. These characteristics collectively contributed to its effectiveness in managing the complexities of the network telescope data and delivering reliable and accurate predictions for the packet count.

The application of the Random Forest Regressor in predicting the packet count of network telescope data was driven by its robustness in handling intricate, high-dimensional datasets and its adeptness in managing noise and outliers. With the network telescope data often characterized by a complex interplay of various parameters and exhibiting non-linear patterns, the Random Forest Regressor proved instrumental in capturing the nuanced relationships and dependencies within the data. Its ensemble approach effectively mitigated the impact of outliers and noise, ensuring the reliability and accuracy of the packet count predictions. Furthermore, the model's capability to manage a large number of variables without overfitting enabled a comprehensive analysis of the diverse network parameters, empowering the extraction of valuable insights for proactive cybersecurity measures.

By leveraging the strengths of the Random Forest Regressor, the predictive analysis of the packet count in network telescope data yielded actionable intelligence crucial for understanding network activity and preemptively addressing potential cybersecurity threats. The model's ability to decipher non-linear patterns and automatically select pertinent features facilitated a

comprehensive assessment of the complex network parameters, enhancing the accuracy and reliability of the predictions. This strategic utilization of the Random Forest Regressor underscored its instrumental role in deciphering the intricate dynamics of network telescope data, ultimately contributing to a proactive and effective cybersecurity approach.

### 3.3 RNN

Deep learning is a subset of machine learning that focuses on the development and training of artificial neural networks, mimicking the structure and function of the human brain, to solve complex problems. It involves learning representations of data through multiple layers, allowing the algorithm to extract hierarchical features and patterns.

Deep learning differs from traditional machine learning in its ability to automatically learn representations from raw data. Rather than relying on handcrafted features, deep learning models can learn intricate patterns directly from the data, eliminating the need for manual feature engineering. However, deep learning models often require a significant amount of data and computational resources for training, and they can be more complex and challenging to interpret compared to traditional machine learning models.

Dealing with the responder, Deep Learning (DL) offers advantages such as improved anomaly detection, automated feature extraction from raw network data, enhanced predictive analysis, scalability, adaptability to evolving network patterns, robust handling of complex data types, and real-time monitoring capabilities. These benefits collectively empower organizations to strengthen network security, detect potential threats more effectively, and gain comprehensive insights into network behavior, facilitating prompt responses to security incidents and enhancing overall network security measures.

Recurrent Neural Networks (RNNs) represent a specialized category of artificial neural networks crafted to proficiently handle sequential data processing. Distinguished from conventional feedforward neural networks, RNNs integrate an intrinsic memory mechanism, facilitating the assimilation of temporal relationships and intricate patterns within sequences. This fundamental attribute endows RNNs with exceptional capabilities for managing time-series data, natural language processing endeavors, and various other applications concerning sequential data

analysis.

RNNs are characterized by their recurrent connections, facilitating the retention of prior input information and enabling the manifestation of dynamic temporal behavior. This architectural design consists of interconnected nodes organized in a recurrent sequence, facilitating the effective processing of sequential data inputs. With each time step, the RNN integrates an input and a hidden state, which undergoes updates based on the current input and the preceding hidden state. This recurrent structure is instrumental in preserving crucial information about preceding inputs, essential for capturing temporal dependencies within sequential data. The RNN generates its output either from the final hidden state or at each time step, depending on the specific configuration of the RNN architecture.

In the sphere of active network telescope packet prediction and time series analysis, Recurrent Neural Networks (RNNs) offer discernible advantages over traditional Artificial Neural Networks (ANNs). Notably, RNNs exhibit remarkable proficiency in managing time-dependent data through their ability to retain and utilize information from preceding time steps. This capability allows RNNs to effectively capture intricate temporal patterns and dependencies within sequential data, surpassing the capabilities of ANNs in this context. Additionally, RNNs demonstrate exceptional competence in processing sequences of varying lengths, making them highly adaptable for comprehensively analyzing the dynamic characteristics of network traffic data. The inherent memory elements within RNNs facilitate the assimilation of historical context into ongoing analyses, enabling the comprehensive capture of long-term dependencies and trends within time series data. This quality is pivotal for ensuring the robustness of network traffic analysis. Furthermore, RNNs exhibit notable efficacy in real-time analysis and expeditious anomaly detection, thereby enabling the rapid identification of potential network threats as they emerge. Collectively, these advantages firmly establish RNNs as the preferred choice for addressing the intricate requirements of active network telescope packet prediction.

They play a critical role in the active network telescope's packet prediction and time series analysis by leveraging their ability to capture temporal dependencies and patterns within sequential data. This allows them to effectively model and predict future network packet behavior, facilitating the identification of potential network anomalies, intrusions, or threats in

real time. By utilizing RNNs, the active network telescope can enhance its predictive capabilities, enabling proactive measures to be taken for ensuring network security and stability. At each time step $t$, the RNN takes an input vector $x_t$ and the previous hidden state vector $h_{t-1}$ and computes the current hidden state $h_t$ using the following equations:

$$h_t = \sigma\left(W_{hh} h_{t-1} + W_{xh} x_t + b_h\right) \quad \textbf{(4)}$$

where:

- $h_t$ is the current hidden state at time $t$.
- $W_{hh}$ is the weight matrix for the recurrent connections.
- $W_{xh}$ is the weight matrix for the connections from the input $x_t$ to the hidden state $h_t$.
- $b_h$ is the bias vector for the hidden state.
- σ is the activation function, often the hyperbolic tangent or the rectified linear unit (ReLU) function.

The output at time $t$ can be computed based on the hidden state $h_t$ using:

$$y_t = softmax(W_{hy} h_t + b_y) \quad \textbf{(5)}$$

where:

- $y_t$ is the output at time $t$.
- $W_{hy}$ is the weight matrix connecting the hidden state to the output.
- $b_y$ is the bias vector for the output.
- *softmax* is the activation function that squashes the output into a probability distribution over different class.

Normalization and smoothing are essential preprocessing techniques for Recurrent Neural Networks (RNNs). Normalization, such as min-max scaling, is crucial for ensuring that input features are on a similar scale, preventing certain inputs from dominating the training process. This enables the RNN to converge efficiently during training and helps mitigate the vanishing or exploding gradient problems. Smoothing, often achieved through techniques like moving

averages, is beneficial for removing noise and irregularities from the time series data, allowing the RNN to identify underlying patterns and trends more effectively, thereby enhancing the network's ability to capture long-term dependencies and make more accurate predictions.

The dataset utilized for analysis comprises comprehensive network telescope data, encompassing various features, namely 'day', 'month', 'year', 'country_count', 'ip_count', 'aso_count', 'asn_count', 'proxy_count' along with the target variable 'packet_count'. Prior to model training, the dataset was divided into distinct subsets, namely the training set, the validation set, and the testing set. The training set consisted of 700 data points, while the validation and testing sets encompassed 30 data points each, ensuring a robust evaluation of the model's predictive capabilities.

Notably, the training set covered a significant period, accounting for the initial 700 entries in the dataset. Following the training set, the subsequent 30 data points were allocated to the validation set, representing a specific time segment for validating the model's performance. Similarly, the testing set, comprising the final 30 data points, served to assess the model's predictive accuracy on unseen data, thereby gauging its generalization capability beyond the trained dataset.

Furthermore, the model architecture was meticulously designed, integrating three layers of the Simple RNN module, with diverse memory units, to capture the temporal patterns and intricate dependencies present in the network telescope data. The model was systematically trained and validated across 500 epochs, with periodic checkpoints for monitoring essential performance metrics, including the loss function, mean absolute error (MAE), mean squared error (MSE), and correlation coefficients for both the training and validation sets.

Consequently, the visualizations illustrating the model's learning progression and its predictive prowess were meticulously analyzed and documented for each epoch. These visual representations not only underscored the model's capacity to discern complex temporal patterns but also highlighted its ability to make accurate predictions in the context of network telescope data analysis. The comprehensive evaluation and analysis provided valuable insights into the efficacy and practical application of RNN models, emphasizing their significance in comprehending and forecasting intricate network behavior and trends.

## 3.4 LSTM

Long Short-Term Memory (LSTM) is a type of recurrent neural network (RNN) architecture designed to address the limitations of traditional RNNs in capturing long-range dependencies and overcoming the vanishing gradient problem. LSTMs are particularly effective for processing and making predictions based on time series or sequential data.

Unlike traditional RNNs, LSTMs incorporate a more complex structure, including a memory cell and various gates such as the input gate, forget gate, and output gate. These components enable LSTMs to selectively retain or forget information over long sequences, allowing them to capture and learn dependencies over extended time intervals. The memory cell serves as a conveyor belt, allowing information to flow through time and ensuring that relevant information can be preserved for long periods.

The key differences between LSTMs and traditional RNNs lie in the architecture's ability to handle long-range dependencies and mitigate the vanishing gradient problem. LSTMs achieve this by allowing the network to retain important information over extended time intervals and preventing the loss of information that occurs in standard RNNs, thereby enabling more effective learning and prediction in tasks involving sequential or time series data.

LSTM networks, a type of RNN, do not require explicit feature selection due to their inherent memory cells that allow them to capture temporal dependencies and patterns automatically. Their architecture enables the processing of sequences of varying lengths without the need for manual feature selection, making them well-suited for handling complex temporal data. The automatic feature selection capability simplifies the modeling process and facilitates efficient analyses of sequential data.

The inclusion of features with relatively low correlation values remains a crucial aspect of the model building process for packet prediction in network telescope data. While correlation serves as a metric for linear relationships, the intricate capability of deep learning models lies in their ability to uncover and learn complex non-linear patterns within the data. This characteristic allows these models to leverage even seemingly less correlated features, extracting valuable

insights that might not be apparent through traditional correlation analysis alone.

In the packet prediction model for network telescope data, all features were considered during the model building process even though they possess less correlation values, to capture the intricate dependencies and patterns, facilitating more accurate predictions.

While model building, the concept of a moving window is integrated to the data preparation process for training, validation, and testing the LSTM model. Specifically, the moving window technique involves systematically shifting the window along the time series data, generating multiple subsequences for training the model. This approach enables the LSTM model to learn from sequential patterns and temporal dependencies within the data, facilitating more robust and comprehensive predictions.

Implemented through the 'prepare_input_data' function, the moving window method segments the dataset into smaller windows of a specified size. These windows consist of sequential data points, ensuring that the model can learn from the historical information within each window and predict the subsequent data point. The size of the window, often referred to as the window size, directly influences the amount of historical information that the model can utilize for prediction.

Furthermore, the utilization of the moving window technique is essential for capturing the temporal dynamics and patterns embedded within the network telescope data. By systematically shifting the window along the dataset, the LSTM model can effectively capture short-term dependencies and long-term trends, enabling it to make accurate and reliable predictions based on the sequential information.

The incorporation of the moving window concept in the data preparation process underscores the significance of temporal context and sequential dependencies in the LSTM model training, thereby enhancing the model's capacity to discern intricate patterns and trends within the network telescope data. This approach ensures that the model can effectively capture the temporal dynamics and make informed predictions based on the historical context encoded within each moving window.

Moving window is used for predicting a packet count on a particular day depending on the

desired limit of past information. A window with limit 7 is described as the $Y_{day_n}$ describes the packet count value at n[th] day which is predicted based on the non-linear function defined on the information obtained from the desired prior days, as $X_{day_{n-1}}$ encloses a multidimensional array of all the features i.e., timestamp, country count,ip count, asn count, aso count, proxy count and packet count on n-1[th] day.

$$f\left(X_{day_1}, X_{day_2}, X_{day_3}, X_{day_4}, X_{day_5}, X_{day_6}, X_{day_7}\right) = Y_{day_8} \quad \textbf{(6)}$$

$$f\left(X_{day_2}, X_{day_3}, X_{day_4}, X_{day_5}, X_{day_6}, X_{day_7}, X_{day_8}\right) = Y_{day_9} \quad \textbf{(7)}$$

Similarly, the general formula is given as,

$$f\left(X_{day_{n-7}}, X_{day_{n-6}}, X_{day_{n-5}}, X_{day_{n-4}}, X_{day_{n-3}}, X_{day_{n-2}}, X_{day_{n-1}}\right) = Y_{day_n} \quad \textbf{(8)}$$

Where,

$f\left(X_{day_{n-7}}, X_{day_{n-6}}, X_{day_{n-5}}, X_{day_{n-4}}, X_{day_{n-3}}, X_{day_{n-2}}, X_{day_{n-1}}\right)$ is the non-linear function defined on the information obtained from the past days

The Long Short-Term Memory (LSTM) network is a type of recurrent neural network (RNN) that is capable of learning long-term dependencies. The key formulas used in the LSTM architecture includes several gates that control the flow of information:

$$i_t = \sigma\left(W_{xi} x_t + W_{hi} h_{t-1} + W_{ci} c_{t-1} + b_i\right)$$

**(9)**

$$f_t = \sigma\left(W_{xf} x_t + W_{hf} h_{t-1} + W_{cf} c_{t-1} + b_f\right)$$

***(10)***

$$C_t = f_t \odot C_{t-1} + i_t \odot tanh\left(W_{xc} x_t + W_{hc} h_{t-1} + b_c\right) \quad \textbf{(11)}$$

$$o_t = \sigma\left(W_{xo} x_t + W_{ho} h_{t-1} + W_{co} c_t + b_o\right)$$

***(12)***

$$h_t = o_t \odot tanh\left(C_t\right)$$

***(13)***

where:

$i_t$ is the input gate at time t,

$f_t$ is the forget gate at time t,

$C_t$ is the cell state at time t,

$o_t$ is the output gate at the time t,

$h_t$ is the hidden state at time t,

$x_t$ is the input at time step t,

$h_{t-1}$ is the hidden state at the previous time step,

$C_{t-1}$ is the cell state at the previous time step,

$W$ and $b$ are the weight matrices and bias vectors, respectively, for the different gates and states,

σ represents the sigmoid activation function,

⊙ represents element-wise multiplication.

This governs the flow of information through the LSTM network, enabling it to capture and process long-term dependencies and patterns in sequential data.

### 3.4.1 Hyperparameter tuning

Hyperparameter tuning is a pivotal practice focusing on the refinement of crucial parameters that lie outside the model's learning process. These parameters wield substantial influence over the model's learning trajectory and its ultimate performance. A lucid comprehension and adept explanation of hyperparameter tuning serve as indicators of a thorough understanding of the complexities inherent in the development of DL models.

Hyperparameter tuning becomes indispensable to ensure accurate and efficient predictions, considering the complexity and dynamic nature of network data. Given the importance of predicting incoming packet counts for effective threat detection and network monitoring, tuning specific hyperparameters is crucial for enhancing the model's predictive capabilities and overall performance.

Firstly, experimenting with various optimizers such as Adam, RMSprop, Adagrad, Momentum with SGD is crucial for fine-tuning the learning process. Each optimizer has its own set of hyperparameters that significantly influence the model's convergence speed and stability during training. By varying the optimizers, the network telescope can potentially achieve faster convergence, improved stability, and better predictive accuracy, allowing for more efficient threat detection and network monitoring capabilities. Secondly, adjusting the window size for the LSTM model enables the network telescope to capture short-term and long-term patterns in the incoming packet data more effectively. By experimenting with different window sizes, the model can potentially capture intricate temporal dependencies within the network data, leading to more accurate predictions and enhanced threat detection capabilities.

Furthermore, exploring the impact of changing the LSTM architecture, including the number of LSTM layers and memory units, can significantly influence the model's capacity to capture

complex relationships within the packet data. Increasing the number of LSTM layers and memory units can enhance the network telescope's ability to learn intricate patterns and dependencies, potentially leading to more accurate predictions and improved threat detection capabilities.

Effective hyperparameter tuning in the context of the network telescope's packet prediction not only enhances the model's predictive accuracy but also plays a crucial role in optimizing the network's threat detection and monitoring capabilities. By fine-tuning hyperparameters related to optimizers, window size, and LSTM architecture, the network telescope can achieve more accurate and efficient predictions, leading to enhanced security measures and improved management of network activities.

# CHAPTER 4

# RESULTS AND DISCUSSION

In this section, we delve into the results and analysis of different models applied to predict packet count in the context of network telescope data. The ARIMA model is scrutinized through residual analysis, shedding light on potential anomalies and areas for improvement. The Random Forest model is evaluated based on various performance metrics, revealing its strengths and limitations in predicting packet count. The Recurrent Neural Network (RNN) faces challenges in capturing intricate patterns, and its limitations are explored. The Long Short-Term Memory (LSTM) model undergoes comprehensive analysis, considering various optimizers, window sizes, layer configurations, and memory units. The performance metrics and graphs offer valuable insights into the effectiveness of each model, guiding potential refinements for more accurate predictions in the dynamic network telescope environment.

## 4.1 ANALYSIS

Residual analysis plots provide valuable insights into the performance of the ARIMA model and can guide further adjustments or improvements to enhance the model's accuracy. Certainly, let's go through the interpretation of the residual analysis plots using the provided residual values.

From Figure 4.1, the scatter plot depicts the relationship between the index and the corresponding residuals. From the plot, we can observe a fluctuating pattern, indicating that the residuals don't follow any discernible trend. However, the presence of outliers at certain indices suggests potential anomalies in the data.

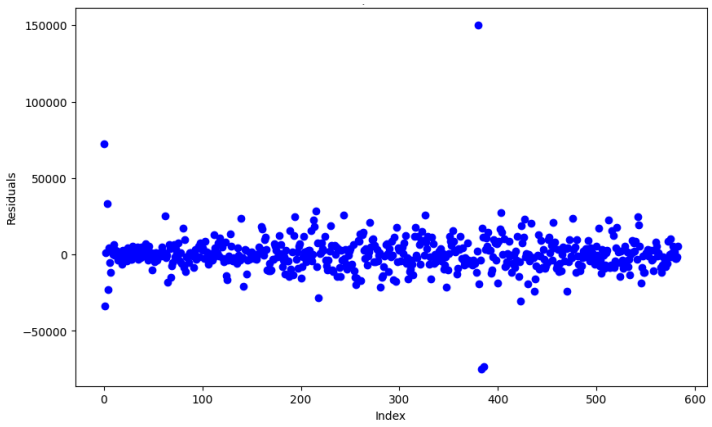



*Figure 4.1 Scatter plot of Residuals*

From Figure 4.2, the density plot showcases the distribution of the residuals. It appears to be approximately normally distributed, centered around zero, which is an encouraging sign. However, the long tails on both sides of the plot imply some extreme values in the residuals.

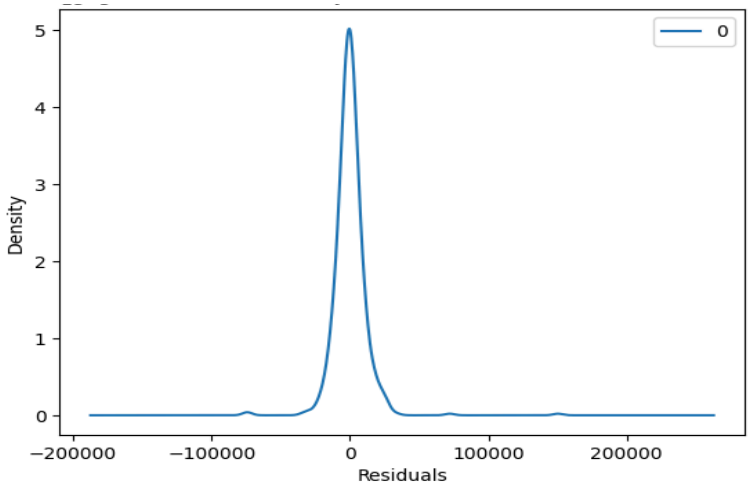



*Figure 4.2 Density plot of Residuals*

From Figure 4.3, the autocorrelation plot is used to identify any significant correlation between the residuals at different lags. In this case, the plot indicates that there might be some autocorrelation present, which suggests that the model might not capture all the underlying patterns in the data.

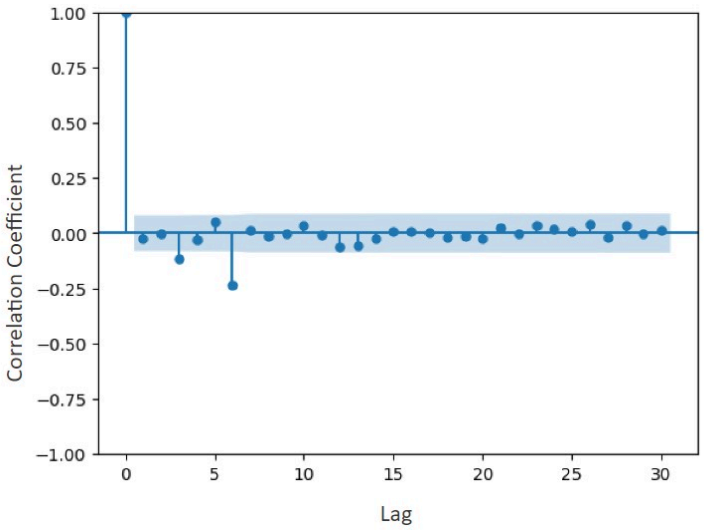



*Figure 4.3 Autocorrelation plot of Residuals*

From Figure 4.4, the Q-Q plot compares the distribution of the residuals to a normal distribution. Here, the residuals seem to follow the diagonal line, implying that they approximately follow a normal distribution. However, the deviations at the tails suggest that the residuals are not entirely normally distributed, potentially indicating the need for further model refinement.

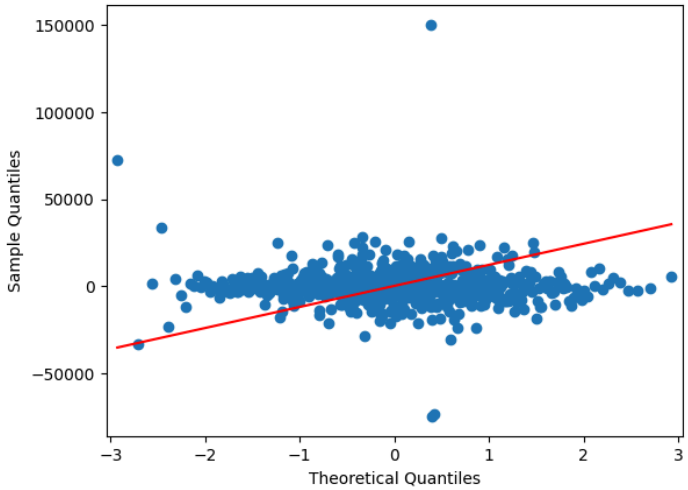



*Figure 4.4 Q-Q plot of Residuals*

Ljung-Box Test Results computed p-value indicate that there is no evidence of significant autocorrelation in the residuals of the ARIMA (5, 1, 0) model. This suggests that the model adequately captures the temporal dependencies present in the data.

Durbin-Watson test statistic, which assesses the presence of autocorrelation in the residuals, has a value of 1.98934049. This value falls close to 2, indicating that there is no significant autocorrelation detected.

Analyzing the provided residuals data as a result of the ARIMA model, we can observe that the values fluctuate significantly with insignificant autocorrelation, ranging from negative to positive values. This suggests that the ARIMA model may not be capturing all the underlying patterns or features within the data, leading to residual fluctuations.

Furthermore, the presence of large positive and negative values in the residuals indicates that the model might be underfitting or overfitting the data, as the predictions are deviating significantly from the actual values. This could be a result of the model's inability to capture the complex relationships within the dataset, indicating the need for further adjustments or improvements to the model's parameters.

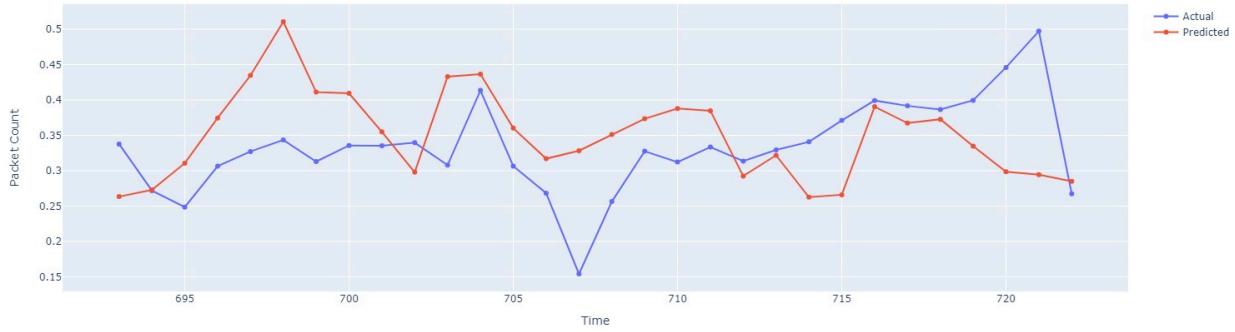



*Figure 4.5 Actual vs predicted graph on testing data to predict packet count by random forest algorithm*

Based on the performance evaluation metrics obtained from the Random Forest model from Figure 4.5, the MSE value of 0.0076 indicates the average of the squared differences between the predicted and actual `packet_count_pred` values, reflecting the model's overall prediction accuracy. Lower values of MSE signify better predictive performance. With an MAE value of 0.0699, the model's average absolute difference between predicted and actual values is relatively moderate. This metric is crucial in understanding the model's precision in predicting the `packet_count_pred` values. The MAPE value of 0.2258 represents the average percentage difference between the predicted and actual `packet_count_pred` values. This metric helps gauge the overall accuracy of the model's predictions, with lower values indicating more accurate

predictions. With an RMSE value of 0.0872, the model's overall predictive error is relatively moderate. RMSE represents the square root of the average of the squared differences between predicted and actual `packet_count_pred` values, providing an understanding of the model's prediction accuracy. The CC value of 0.0597 suggests a weak positive linear relationship between the predicted and actual `packet_count_pred` values. A low CC value indicates that the model might not fully capture the linear relationship between the predicted and actual values. The MASE value of 1.5422 provides insight into the model's predictive accuracy relative to a naive baseline model. A value greater than 1 suggests that the model performs worse than a naive model, indicating room for improvement. The R2 value of -0.8438 signifies the proportion of the variance in the `packet_count_pred` values that the model is unable to explain. A negative R2 value indicates that the model's predictions are worse than simply using the mean of the observed data. Overall, the model exhibits limitations in accurately predicting values, lagging with potential areas for refinement and optimization to enhance its predictive capabilities.

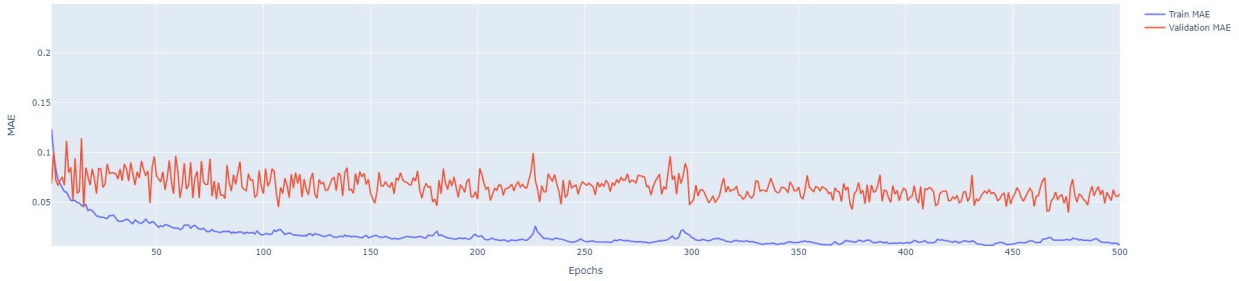



*Figure 4.6 Loss vs Epoch graph for all ports by RNN*

Based on the observed loss-epoch graph of Figure 4.6, it is evident that the recurrent neural network (RNN) encounters limitations even after an exhaustive 500-epoch analysis. The complexity of the packet prediction pattern appears to surpass the learning capacity of the RNN model. This constraint primarily stems from the inherent architectural limitations of the RNN, notably the absence of a prominent forget gate mechanism. Consequently, the RNN model fails to adequately prioritize recent information over historical data as shown in Figure 4.7, thereby impeding its proficiency in capturing and comprehending intricate data patterns.

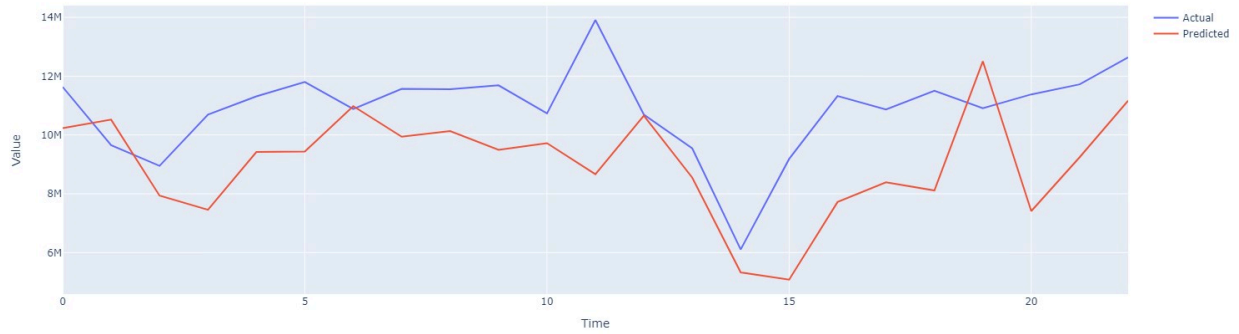



*Figure 4.7 Actual vs predicted graph on testing data to predict packet count by RNN*

| METRIC | VALIDATION | TESTING |
|---|---|---|
| MAE | 0.05186 | 0.07885 |
| MSE | 0.00189 | 0.00658 |
| MAPE | 13.13467 | 21.51264 |
| RMSE | 0.04346 | 0.08114 |
| CC | 0.73390 | 0.50192 |
| MASE | 1.63400 | 1.63755 |

*Table 4.1 Performance measure on validation and testing data to predict packet count by RNN*

Despite displaying promising results in the validation phase in Table 4.1, the model's performance deteriorates notably when applied to unseen testing data. While the evaluation metrics suggest a favorable outcome during the validation process, the model fails to generalize effectively to the broader testing dataset. Specifically, the observed disparity between the predicted and actual values, as illustrated in the actual versus predicted graph, reveals a significant deviation. Although the model captures the underlying patterns present in the original dataset, its inability to accurately predict values within an acceptable margin of error renders it unsuitable for the data at hand. Furthermore investigation and potential recalibration of the model parameters or architecture may be necessary to enhance its predictive capabilities and improve its performance on unseen data.

The Table 4.2 encloses the evaluation metrics of various optimizers like Adam, RMSprop, Adagrad, Momentum with SGD with testing data of port 443 of the LSTM model.

| METRIC | Momentum with SGD | Adagrad | RMSprop | Adam |
|---|---|---|---|---|
| MAE | 0.08590 | 0.13425 | 0.11351 | 0.11079 |
| MSE | 0.01712 | 0.02868 | 0.00693 | 0.00677 |
| MAPE | 14.65831 | 24.81417 | 15.00568 | 15.30818 |
| RMSE | 0.13084 | 0.16936 | 0.08327 | 0.08231 |
| CC | 0.07010 | -0.60388 | 0.79221 | 0.77465 |
| MASE | 2.02948 | 3.19822 | 1.68848 | 1.69423 |

*Table 4.2 Performance measure on various optimizers on testing data to predict packet count by LSTM*

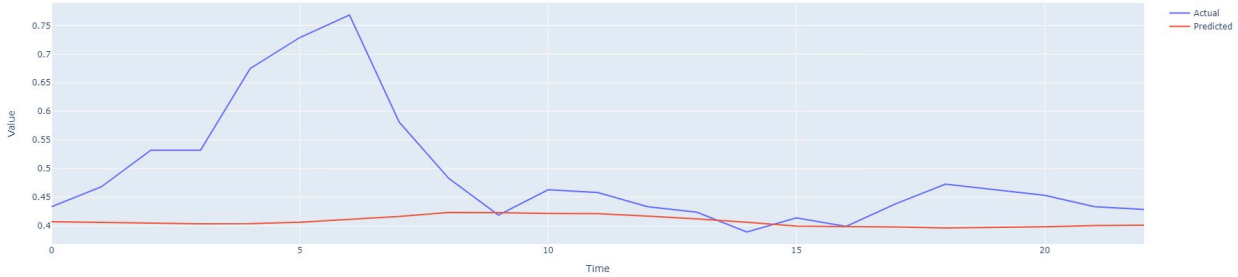



*Figure 4.8 Actual vs predicted graph of Momentum with Stochastic gradient descent optimizer on testing data by LSTM*

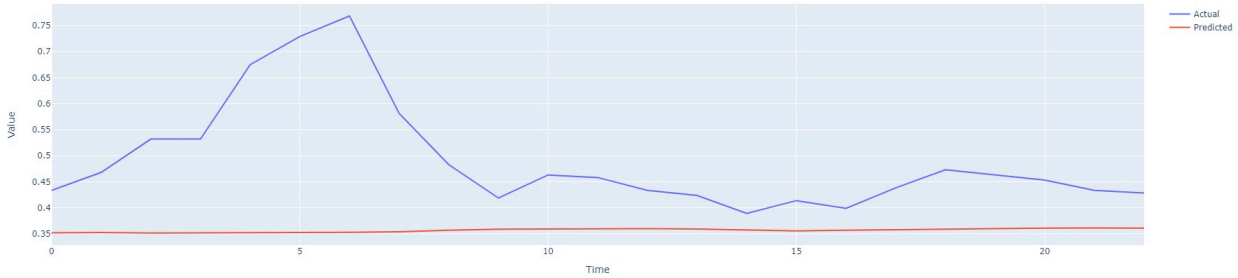



*Figure 4.9 Actual vs predicted graph of Adagrad optimizer on testing data by LSTM*

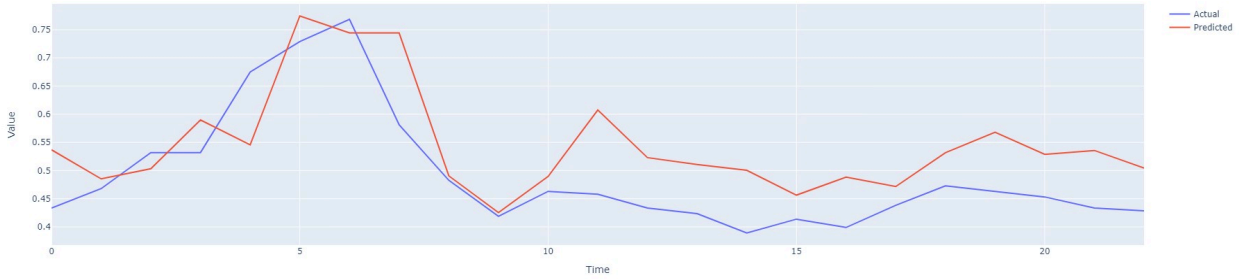



*Figure 4.10 Actual vs predicted graph of RMSprop optimizer on testing data by LSTM*

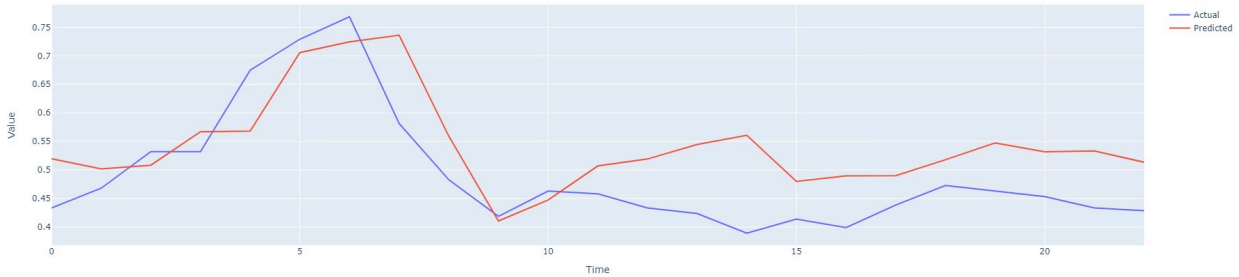



*Figure 4.11 Actual vs predicted graph of Adam optimizer on testing data by LSTM*

Based on the observed Figures 4.8, 4.9, 4.10, 4.11, it is evident that the Momentum with SGD and Adagrad optimizers demonstrate limited learning capabilities over time. Conversely, notable performance improvements were discerned with the RMSprop and Adam optimizers. These findings underscore the significance of selecting appropriate optimizers, with RMSprop and Adam exhibiting superior convergence properties and enhanced learning dynamics, thereby contributing to improved model performance and predictive accuracy over time.

The optimization algorithm used during training, such as stochastic gradient descent (SGD) or Adam, involves randomness in the selection of data batches and the updates to the model's parameters. This randomness can lead to different paths of convergence during each training session. Adam and RMSprop consistently demonstrate superior performance in terms of lower Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Scaled Error (MASE) values. They also exhibit a stronger linear relationship, as indicated by the higher Correlation Coefficient (CC) values, suggesting their reliability in predicting the target variable. Adagrad, while displaying relatively higher error metrics such as MAPE and MASE, maintains a reasonable level of performance but falls slightly behind in comparison to Adam and RMSprop. Momentum with SGD, while showcasing

competitive results, demonstrates a relatively less consistent performance across the evaluated metrics. These insights underscore the importance of selecting an appropriate optimizer, with Adam and RMSprop standing out as the top-performing choices for this specific dataset and modeling task. Overall Adam performs well, hence its employed in the prediction of all ports data.

| WINDOW SIZE | MSE |
|---|---|
| 1 | 0.00407 |
| 2 | 0.00400 |
| 5 | 0.00717 |
| 6 | 0.00590 |
| 7 | 0.0067 |
| 10 | 0.03107 |

*Table 4.3 Loss occurred on various window size on testing data to predict packet count by LSTM*

From Table 4.3, the results showcase the outcomes of hyperparameter tuning conducted for different window sizes, aiming to predict the packet count precisely on a specific day, precisely 30 days ahead. While showcasing promising results with minimized loss, this approach is not without limitations, as the extended duration often leads to the emergence of new patterns caused by the continually evolving Internet background radiation. Consequently, to address this constraint, a modified strategy was implemented, focusing on a shorter time frame of seven days. Employing a window size of seven, this revised method integrated information spanning the preceding seven days to accurately forecast the packet count for the subsequent eighth day. This adaptive approach not only mitigates the impact of evolving patterns but also enhances the accuracy of predictions, ensuring a more robust and reliable predictive framework for the dynamic network telescope data.

| S. No | No of Layer | Memory units | MAE | MSE | MAPE | RMSE | CC | MASE |
|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 64,64 | 0.04626 | 0.00168 | 13.17872 | 0.04104 | 0.79884 | 1.49687 |
| 2 | 2 | 64,32 | 0.03882 | 0.00079 | 8.54110 | 0.02825 | 0.72316 | 0.89907 |
| 3 | 2 | 32,32 | 0.05671 | 0.00206 | 11.85081 | 0.04544 | 0.69128 | 1.50634 |
| 4 | 3 | 64,64,64 | 0.04561 | 0.00234 | 15.29238 | 0.04844 | 0.66191 | 1.69508 |
| 5 | 3 | 64,64,32 | 0.05611 | 0.00311 | 16.72254 | 0.05580 | 0.59699 | 2.14496 |
| 6 | 3 | 64,32,32 | 0.03935 | 0.00070 | 8.78965 | 0.02664 | 0.80218 | 1.03706 |
| 7 | 3 | 64,32,16 | 0.04638 | 0.00197 | 13.29350 | 0.04446 | 0.57122 | 1.69112 |

*Table 4.4 Performance measure on various LSTM layers and memory units on testing data to predict packet count by LSTM*

From the given results of Table 4.4, the analysis of the LSTM model configurations of 30 days ahead reveals the number of layers does not consistently influence the model's performance metrics, suggesting that additional layers may not always lead to improved predictive accuracy. Different memory unit configurations exhibit noticeable variations in performance, with certain configurations, such as "64,32" and "64,64,64," demonstrating competitive performance in capturing intricate data patterns. There appears to be a trade-off between model complexity and performance, as models with simpler configurations, like "2" layers and "32,32" memory units, show competitive results, highlighting the significance of a balanced model architecture. Models with more compact configurations, such as "64,32,32" and "64,32,16," showcase favorable performance metrics, emphasizing the potential effectiveness of streamlined architectures in achieving efficient and accurate packet count predictions.

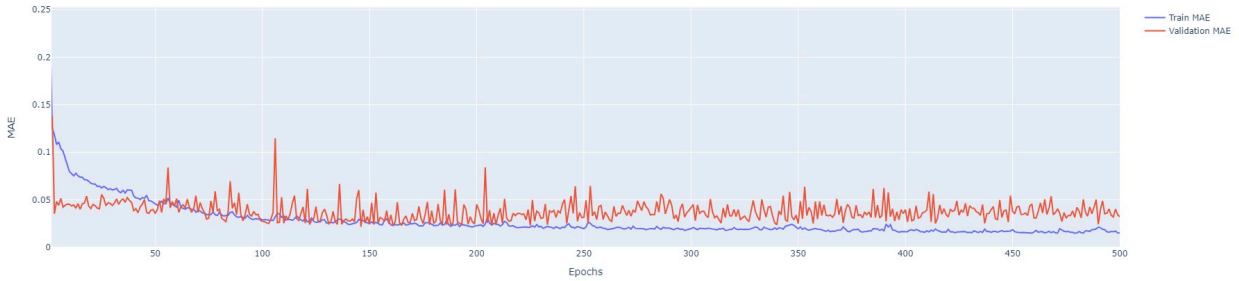



*Figure 4.12 Loss vs Epoch graph for all ports by LSTM*

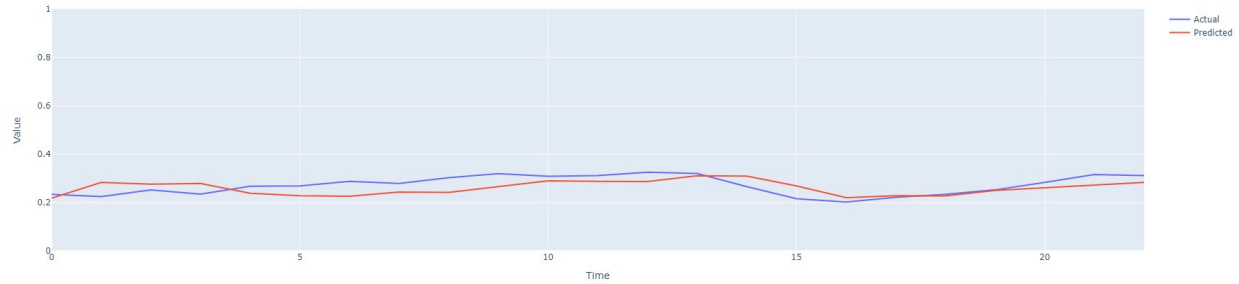



*Figure 4.13 Actual vs predicted graph on validation data to predict packet count by LSTM*

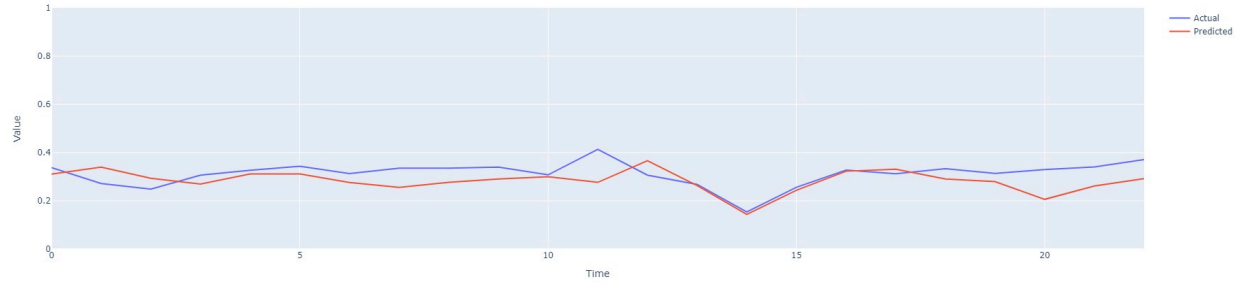



*Figure 4.14 Actual vs predicted graph on testing data to predict packet count by LSTM*

Based on the refined specifications, the optimal three-layer configuration with memory units set at "64,32,32," in conjunction with the Adam optimizer and a window size of 7, was employed for the purpose of forecasting the subsequent day's packet count values. The evaluation of the Epoch versus loss graph facilitated the identification of the most effective epoch count, revealing a notably favorable model performance at epoch 100 is shown Figure 4.12. As such, this pivotal observation guided the selection of the epoch count, affirming its critical role in ensuring the model's accuracy and reliability in validating as shown in Figure 4.13 and predicting packet counts as shown in Figure 4.14 within the network telescope domain.

## 4.2 INFERENCE

The evaluation of multiple models, including ARIMA, Random Forest, RNN, and LSTM, has provided valuable insights into the strengths and limitations of each approach in predicting packet counts within the network telescope domain. The ARIMA model, while capturing temporal dependencies, exhibits fluctuations and anomalies, suggesting a potential need for further model refinement. The Random Forest model demonstrates moderate predictive accuracy but exhibits limitations, indicating the necessity for parameter tuning to enhance its performance. The RNN encounters challenges in comprehending complex patterns, hindering its proficiency in capturing intricate data relationships, while the LSTM model, particularly with optimized configurations, demonstrates the ability to capture nuanced data patterns effectively, highlighting the significance of appropriate model architecture and parameter selection. Notably, the LSTM model's refined three-layer configuration with memory units set at "64,32,32," in conjunction with the Adam optimizer and a window size of 7, stands out as the optimal configuration for accurately forecasting packet counts within the network telescope domain. Overall, these results emphasize the critical role of refined model architectures, optimizer selection, and hyperparameter tuning in achieving accurate and reliable predictions, underscoring the importance of leveraging sophisticated deep learning models for robust analysis within the network telescope framework.

# CONCLUSION

In conclusion, the conducted analysis and evaluation of diverse models in the context of network telescope data prediction reveal promising capabilities, yet also highlight existing limitations. While models such as ARIMA, Random Forest, RNN, and LSTM demonstrate varying degrees of predictive accuracy, each approach faces its own challenges in capturing the dynamic and evolving nature of network activity. Despite the meticulous validation processes employed, it is imperative to acknowledge the persistent presence of residual anomalies and potential data fluctuations that can influence the precision of predictions. Moreover, the ever-evolving landscape of cyber threats necessitates continuous adaptation and refinement of modeling techniques to effectively anticipate and counter the emerging security challenges. As such, the study underscores the critical need for ongoing research and development in the field of network telescope data analysis, emphasizing the importance of integrating sophisticated methodologies to ensure robust and reliable predictive frameworks that align well with the evolving demands of modern cybersecurity.

# BIBLIOGRAPHY

[1] Moore, D., Shannon, C., Voelker, G. M, & Savage, S. (2004). Network Telescopes: Technical Report. UC San Diego: Department of Computer Science & Engineering, 2004-07-07, UC San Diego.

[2] Yu Jin, György Simon, Kuai Xu, Zhi Li Zhang, Vipin Kumar, Gray's anatomy: Dissecting scanning activities using IP gray space analysis, 2nd Workshop on Tackling Computer Systems Problems with Machine Learning Techniques, Sys ML 2007, co-located with NSDI 2007 - Cambridge, United States, Apr 10 2007.

[3] Félix Iglesias and Tanja Zseby, Entropy-Based Characterization of Internet Background Radiation, OPEN ACCESS entropy, Published: 31 December 2014, Entropy 17(1): 74-101 (2015).

[4] Farooq Shaikh, Elias Bou-Harb, Jorge Crichigno, Nasir Ghani, A Machine Learning Model for Classifying Unsolicited IoT Devices by Observing Network Telescopes, Published in: 2018 14th International Wireless Communications & Mobile Computing Conference (IWCMC), Conference Location: Limassol, Cyprus, June 2018, IEEE.

[5] Nipun Ramakrishnan; Tarun Soni, Network Traffic Prediction Using Recurrent Neural Networks, Published in: 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA) Date of Conference: 17-20 December 2018, Conference Location: Orlando, FL, USA, December 2018, IEEE.

[6] Sanjay Kumar, Harald Vranken, Joost van Dijk, Timo Hamalainen, Deep in the Dark: A Novel Threat Detection System using Darknet Traffic, Published in: 2019 IEEE International Conference on Big Data (Big Data), Date of Conference: 09-12 December 2019, Conference Location: Los Angeles, CA, USA, IEEE.

[7] Olivier Cabana; Amr M. Youssef; Mourad Debbabi; Bernard Lebel; Marthe Kassouf; Ribal Atallah; Basile L. Agba, Threat Intelligence Generation Using Network Telescope Data for Industrial Control Systems, Published in: IEEE Transactions on Information Forensics and

Security ( Volume: 16), Page(s): 3355 – 3370, IEEE, Date of Publication: 07 May 2021.

[8] Michalis Kallitsis, Vasant G. Honavar, Rupesh Prajapati, Dinghao Wu, John Yen: Zooming Into the Darknet: Characterizing Internet Background Radiation and its Structural Changes, arXiv:2108.00079v2 [cs.CR] 5 Aug 2021.

[9] Almomani, A. Darknet traffic analysis, and classification system based on modified stacking ensemble learning algorithms. Inf Syst E-Bus Manage (2023).